IP Report - Probabilistic Computer Design

Anurag Gulati (anurag20176@iiitd.ac.in)
Aryman Srivastava (aryman20184@iiitd.ac.in)
Ullas Pai (ullas20255@iiitd.ac.in)
Varun Muttepawar (varun20259@iiitd.ac.in)

Winter 2024 Semester

1 Weight Matrix and Bias Vector Training

Typically, the weight matrix and bias vectors in a PSL network are derived analytically using techniques from linear algebra. However, recent works [3] have proposed training algorithms which enable to network to learn its parameters by minimizing an objective function.

We use the training algorithm proposed in [3] and derive the weights and biases for different network functionalities by tuning the hyperparameters such as learning rate, and regularization. We would like to acknowledge Amina Haroon (aminah@iiitd.ac.in) for her support and expertise on the source code of the training algorithm.

Based on the required functionality (such as AND gate, half-adder, 2-bit full adder, etc.), we can create a bipolar-encoded truth table and use it as the training data for the network. The network treats the rows of truth table as valid states and learns its parameters to maximize the probabilities of these valid states, and by effect, minimize that for invalid states.

During training, we use KL divergence as the objective function which is to be minimized over iterations. In order to speed up the training process, we can set an acceptable threshold for KL divergence, beyond which the training process is terminated.

2 About the Code

2.1 The Pbit Module

This module, as described in [1], importantly, takes enable and bias (z) as input from the fixed-point calculation performed in the PSL module.

Internally, it has an LUT-based Tanh module which takes the bias (z) as is as input. It also has an RNG block initialized from a different seed value for each pbit module instance.

The sum of the LUT-tanh result and random number from the LFSR is calculated and the MSB bit is analysed to output a 1 or a 0 as the output of the pbit module instance. The LFSR was designed to include the zero state.

2.2 The PSL Module

Let us begin with the main module. The "psl" module - having multiplier inputs in1 and in2 (the multiplier and multiplicand, respectively) in output deterministic mode and op (the product) as its input in input deterministic mode, apart from clock, reset, operating mode signal (input or output deterministic) and pending_request signal which are required inputs.

Output deterministic mode - In this mode the multiplicand and multiplier are provided as inputs to the PSL with MODE set to 1'B0. The result from the module is updated every 3 clock cycles.

Input deterministic mode - In this mode the product is only provided as input to the PSL with MODE set to 1'B1. The result from the module is updated every 3 clock cycles.

The Module takes the "fixed" bias vector and weight matrix obtained from training, as parameters to it. The weight matrix has pbit `x` pbit dimension and the bias vector has "pbit" number of elements.

Internally, parallel hardware is allocated for fixed-point adder and multiplier [2] blocks for "dynamic" bias calculation. The source code for these modules (fixed point adder and multiplier) were tweaked for correctness and appropriate use. This bias is provided to the pbit module. It has as many pbit module instances as number of output bits to calculate (i.e, 4 bits for a 2-bit multiplier).

2.3 The Result Interpreter Module

This module is responsible for providing a deterministic result from the result of the PSL Module. It also takes clock, reset and pending_request as inputs apart from psl_out (output of the PSL module).

The result from the PSL module is sampled once all the output pbits (in output determinitic mode) or input pbits (in input deterministic mode) are updated at least once.

It is sampled as many number of times as required for a decision to be made. We chose the decision point to be 20 (i.e, the number of update cycles required

to obtain deterministic result from the multiplier). Once the decision point is attained, the result is "readied" using the valid_res signal.

2.4 The Wrapper Module

This module is the topmost, outermost module which instantiates the PSL and the Result Interpreter Modules aside from being responsible for handling the pending_request signal (which is required by both its module instances).

3 Concluding Remarks - New PSL and FPGA Testing

For the synthesis of every new probabilistic logic circuit, one must train and obtain appropriate weights and biases to feed to the weighted-pbit module (the PSL module) as its parameters and one must specify the number of input pbits (P3), output pbits (P2) and total number of pbits (P) in the outermost module.

Lastly, to test on FPGA - the code has to tested using a vio_wrapper file along with a constraints file. The Vivado project file is provided as an example to test the 2-bit multiplier on FPGA.

References

[1] Pervaiz, A.Z., Sutton, B.M., Ghantasala, L.A., & Camsari, K.Y., (n.d.). Weighted p-Bits for FPGA Implementation of Probabilistic Circuits.

[2] https://github.com/freecores/verilog_fixed_point_math_library

[3] A. Haroon and S. Saurabh, "Image Completion using a Sparse Probabilistic Spin Logic Network," 2022 35th International Conference on VLSI Design and 2022 21st International Conference on Embedded Systems (VLSID), Bangalore, India, 2022, pp. 281-286, doi: 10.1109/VLSID2022.2022.00061.